Advanced Topics in Software Verification

Christian Rinderknecht
Christian.Rinderknecht@tezcore.com
Nomadic Labs (Paris)
28 October 2018

Tezos Masterclass

Introduction

- This lecture proposes a technical overview of several topics in the field of **software verification**.
- It is a mathematical lecture, more specifically, **formal logic** lies at its core.
- We will start with perhaps the most obvious guarantee when running software: the **proof of termination**.
- Once we know that a program terminates on all its inputs, we would like to get an estimation of its running time, depending on the inputs: this is the **analysis of algorithms**.
- Finally, we may want to **optimise a program**. For that, we need to **prove logical properties** about it, and perhaps replace it by a better one, so we need **proofs of equivalence** between two programs.

Termination

- When defining our purely functional language, we could actually have imposed some syntactic restrictions on recursive definitions in order to guarantee the termination of all function calls.
- A well-known class of such terminating functions makes exclusively use of a bridled form of recursion called **primitive recursion**.
- Unfortunately, many useful functions do not fit easily in this framework and, as a consequence, most functional languages leave to the programmers the responsibility to check the termination of their programs.
- **Termination is an undecidable problem**, that is, it is not possible to provide a general criterion for termination, but many standard rules exists that cover many usages.
- Let us review a few techniques.

Termination

- Here is again the definition of the factorial function as a rewite system:

$$\mathsf{fact}(0) \to 1$$

$$\mathsf{fact}(n) \to n \times \mathsf{fact}(n-1)$$

- We could use OCaml to define the factorial, of course, but we want to make it clear that what we demonstrate in this lecture does not depend on OCaml itself, but, instead, applies to a host of functional languages.
- Given a call $\mathsf{fact}(n)$, with $n \geqslant 0$, the rewrites yield a series of calls $\mathsf{fact}(n-1)$, $\mathsf{fact}(n-2)$, etc. until $\mathsf{fact}(0)$.
- Termination here ensues from the **strictly decreasing argument** n, $n-1$, $n-2$ etc. down to 0. (No infinite descent.)

- Consider the following example where $m,n \in \mathbb{N}$:

$$\begin{aligned} \mathsf{ack}(0,n) &\xrightarrow{\theta} n+1 \\ \mathsf{ack}(m+1,0) &\xrightarrow{\iota} \mathsf{ack}(m,1) \\ \mathsf{ack}(m+1,n+1) &\xrightarrow{\kappa} \mathsf{ack}(m,\mathsf{ack}(m+1,n)) \end{aligned}$$

- This is a simplified form of Ackermann's function, an early example of a (total) computable function which is **not primitive recursive**. It makes use of double recursion and grows values as towers of exponents, for example,

$$\mathsf{ack}(4,3) \twoheadrightarrow 2^{2^{65536}} - 3.$$

- It is not obviously terminating, because if the first argument does decrease or remains the same, the second **largely increases**: we need a stronger criterion to prove the termination.

Well-founded and lexicographic orders

- We define a **well-founded order** on a set A as a binary relation $(\succ)$ which does not have any **infinite descending chains**, to wit, there is no $a_0 \succ a_1 \succ \ldots$
- Let us define a well-founded order on **pairs**, called **lexicographic order**. Let $(\succ_A)$ and $(\succ_B)$ be well-founded orders on the sets A and B.
- Then $(\succ_{A\times B})$ defined as follows on $A \times B$ is well-founded:
$$(a_0, b_0) \succ_{A\times B} (a_1, b_1) :\Leftrightarrow a_0 \succ_A a_1 \text{ or } (a_0 = a_1 \text{ and } b_0 \succ_B b_1).$$
- For example, $(\underline{1},7) \prec (\underline{2},3)$, $(1,\underline{7}) \succ (1,\underline{3})$.
- If $A = B = \mathbb{N}$ then $(\succ_A) = (\succ_B) = (>)$.
- To prove that $\mathsf{ack}(m, n)$ terminates for all $m, n \in \mathbb{N}$, first, we must find a well-founded order on the calls $\mathsf{ack}(m,n)$.

- That is to say, the calls must be totally ordered without any infinite descending chain. In this case, a lexicographic order on $(m,n) \in \mathbb{N}^2$ extended to $\mathsf{ack}(m,n)$ works:

$$\mathsf{ack}(a_0,b_0) \succ \mathsf{ack}(a_1,b_1) :\Leftrightarrow a_0 > a_1 \text{ or } (a_0 = a_1 \text{ and } b_0 > b_1).$$

- Clearly, $\mathsf{ack}(0,0)$ is the minimum element.
- Second, we must prove that "ack" rewrites to smaller calls.
- We are only concerned with rules ι and κ.
- With the former, we have $\mathsf{ack}(m+1,0) \succ \mathsf{ack}(m,1)$.
- With the latter, we have

$$\mathsf{ack}(m+1,n+1) \succ \mathsf{ack}(m+1,n)$$
$$\mathsf{ack}(m+1,n+1) \succ \mathsf{ack}(m,p),$$

  for all values p, in particular when $\mathsf{ack}(m+1,n) \twoheadrightarrow p$. □

- In general, the approach often taken to prove termination consists in finding a well-founded order ($\succ$) on the rewritten terms, which is entailed by the rewrite relation ($\rightarrow$), that is,

$$x \rightarrow y \Rightarrow x \succ y.$$

- As we have seen, we do not always need to order the terms x and y: only the calls in them to a given function. These are **dependency pairs**. Calls to other functions can be assumed to be terminating.
- With the factorial and Ackermann's function, we have seen how to deal with one or more integer arguments to these calls.
- What about other data types, like lists and trees?
- One well-founded order for lists (stacks) is the **immediate subterm order**, satisfying

$$x :: s \succ s \quad \text{and} \quad x :: s \succ x.$$

- The immediate subterm order simply means that a list is "greater" than its first element and the rest of the list (called the **tail**).
- It is a special case of the **proper subterm order**, in which a structured value is "greater" than any of its components, wherever they are located.
- These orders are convenient to prove the termination of calls to functions whose definitions are recursive in terms of substructures of their arguments: this is quite a common case.
- For example, let us recall the function catenating a list to another:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t$$
$$\mathsf{cat}(x::s,t) \xrightarrow{\beta} x::\mathsf{cat}(s,t)$$

We simply have

$$x::s \succ s \Rightarrow \mathsf{cat}(x::s,t) \succ \mathsf{cat}(s,t).$$

- Let us try a more complicated example:

$$\begin{aligned} \mathsf{flat}([]) &\xrightarrow{\psi} [] \\ \mathsf{flat}([] :: t) &\xrightarrow{\omega} \mathsf{flat}(t) \\ \mathsf{flat}((x :: s) :: t) &\xrightarrow{\gamma} \mathsf{cat}(\mathsf{flat}(x :: s), \mathsf{flat}(t)) \\ \mathsf{flat}(y :: t) &\xrightarrow{\delta} y :: \mathsf{flat}(t) \end{aligned}$$

- Some evaluations:

$$\begin{aligned} \mathsf{flat}([]) &\twoheadrightarrow [] \\ \mathsf{flat}([[];[[]]]) &\twoheadrightarrow [] \\ \mathsf{flat}([[];[1;[2;[]];3];[]]) &\twoheadrightarrow [1;2;3] \end{aligned}$$

- What "flat" does is flatten a list of lists arbitrarily deep.
- Note that this function cannot be written in OCaml because the lists in question are not homogeneous.

A detailed evaluation:

$$
\begin{aligned}
\mathsf{flat}([];[[1];2];3]) &\xrightarrow{\omega} \mathsf{flat}([[[1];2];3]) \\
&\xrightarrow{\gamma} \mathsf{cat}(\mathsf{flat}([[1];2]),\mathsf{flat}([3])) \\
&\xrightarrow{\gamma} \mathsf{cat}(\mathsf{cat}(\mathsf{flat}([1]),\mathsf{flat}([2])),\mathsf{flat}([3])) \\
&\xrightarrow{\delta} \mathsf{cat}(\mathsf{cat}(1::\mathsf{flat}([]),\mathsf{flat}([2])),\mathsf{flat}([3])) \\
&\xrightarrow{\psi} \mathsf{cat}(\mathsf{cat}([1],\mathsf{flat}([2])),\mathsf{flat}([3])) \\
&\xrightarrow{\delta} \mathsf{cat}(\mathsf{cat}([1],2::\mathsf{flat}([])),\mathsf{flat}([3])) \\
&\xrightarrow{\psi} \mathsf{cat}(\mathsf{cat}([1],[2]),\mathsf{flat}([3])) \\
&\xrightarrow{\delta} \mathsf{cat}(\mathsf{cat}([1],[2]),3::\mathsf{flat}([])) \\
&\xrightarrow{\psi} \mathsf{cat}(\mathsf{cat}([1],[2]),[3]) \\
&\twoheadrightarrow [1;2;3].
\end{aligned}
$$

- The function "cat" is independent of "flat" and we proved its termination in isolation by using the proper subterm order on its first argument.
- Assuming now that "cat" terminates, let us prove the termination of "flat".
- Because the recursive calls to "flat" contain only (list) constructors, we can try to order their arguments.
- The proper subterm order works:

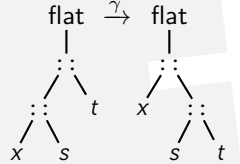
$$\begin{aligned} y :: t &\succ t && \text{(rule } \delta \text{ and rule } \omega \text{ when } y = []) \\ (x :: s) :: t &\succ t && \text{(rule } \gamma) \\ (x :: s) :: t &\succ x :: s && \text{(rule } \gamma) \end{aligned}$$

- Termination ensues. □

- Let us consider now an alternative design for rule γ:

$$\begin{aligned} \mathsf{flat}([]) &\xrightarrow{\psi} [] \\ \mathsf{flat}([] :: t) &\xrightarrow{\omega} \mathsf{flat}(t) \\ \mathsf{flat}((x :: s) :: t) &\xrightarrow{\gamma} \boxed{\mathsf{flat}(x :: s :: t)} \\ \mathsf{flat}(y :: t) &\xrightarrow{\delta} y :: \mathsf{flat}(t) \end{aligned}$$

- This amounts to perform a **right rotation** of the abstract syntax tree of the term:



- Here, the proper subterm order fails:
$$(x :: s) :: t \not\succ x :: (s :: t) = x :: s :: t$$
because $t \not\succ s :: t$.
- We need another method to prove termination: **measures**.
- A measure $\mathcal{M}[\![\cdot]\!]$ is a map from terms to a well-ordered set $(A, \succ)$, which is **monotonically increasing** with respect to the rewrite relation:
$$x \to y \Rightarrow \mathcal{M}[\![x]\!] \succ \mathcal{M}[\![y]\!] \Rightarrow x \succ y.$$
- Actually, as earlier, we will only consider **dependency pairs**, that is, pairs of calls whose first component is the left-hand side of a rule and the second components are the calls in the righ-hand side of the same rule.

- The pairs are
  1. $(\mathsf{flat}([] :: t), \mathsf{flat}(t))_\omega$
  2. $(\mathsf{flat}((x :: s) :: t), \mathsf{flat}(x :: s :: t))_\gamma$
  3. $(\mathsf{flat}(y :: t), \mathsf{flat}(t))_\delta$

  with $y \notin S$, that is, y is not a list.
- We can actually drop the function names, as all the pairs involve "flat".
- A common class of measures are monotonically increasing embeddings into $(\mathbb{N}, >)$, so let us seek a measure satisfying:

$$\begin{aligned} \mathcal{M}[\![(x :: s) :: t]\!] &> \mathcal{M}[\![x :: s :: t]\!] \\ \mathcal{M}[\![y :: t]\!] &> \mathcal{M}[\![t]\!], \qquad \text{if } y \notin S \text{ or } y = []. \end{aligned}$$

- For instance, let us set the following **polynomial measure**:

$$\begin{array}{rcll} \mathcal{M}[\![x :: s]\!] & := & 1+2\cdot\mathcal{M}[\![x]\!]+\mathcal{M}[\![s]\!] & \\ \mathcal{M}[\![y]\!] & := & 0, & \text{if } y \notin S \text{ or } y = []. \end{array}$$

- We have

$$\begin{array}{rcl} \mathcal{M}[\![(x :: s) :: t]\!] & = & 3+4\cdot\mathcal{M}[\![x]\!]+2\cdot\mathcal{M}[\![s]\!]+\mathcal{M}[\![t]\!] \\ \mathcal{M}[\![x :: s :: t]\!] & = & 2+2\cdot\mathcal{M}[\![x]\!]+2\cdot\mathcal{M}[\![s]\!]+\mathcal{M}[\![t]\!] \end{array}$$

- Then

$$\begin{array}{rcl} \mathcal{M}[\![(x :: s) :: t]\!] & > & \mathcal{M}[\![x :: s :: t]\!] \\ \mathcal{M}[\![y :: t]\!] = 1+\mathcal{M}[\![t]\!] & > & \mathcal{M}[\![t]\!] \end{array}$$

- Since $\mathcal{M}[\![x]\!] \in \mathbb{N}$, for all x, these inequalities entail the termination of "flat" (no infinite descent). □

Analysis of algorithms

- The mathematical study of the efficiency of algorithms has been pioneered by Knuth, who called it **analysis of algorithms**.
- Given a function definition,
  1. we define a measure on the arguments, which represents their size;
  2. we define a measure on time, which abstracts the wall clock time;
  3. we express the abstract time needed to compute calls to that function in terms of the size of its arguments: this is the **cost function**.
- The lower the cost, the higher the efficiency.
- For example, for sorting objects, called **keys**, by comparing them,
  1. the input size is the number of keys,
  2. the abstract unit of time is one comparison,
  3. and the cost maps the number of keys to the number of comparisons to sort the keys.

Exact cost

- Rewrite systems enable a rather natural notion of cost for functional programs: it is the number of rewrites to reach the value of a function call, assuming that the arguments are values.
- In other words, the **exact cost** is the number of calls needed to compute the value of a given call.
- For stacks, the measure of the size of the input is the number of items it contains, or **length** of the stack.

- For instance, let us recall the appending of a stack:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t$$
$$\mathsf{cat}(x::s,t) \xrightarrow{\beta} x::\mathsf{cat}(s,t)$$

- We observe that t is invariant, so the cost depends only on the size of the first argument.
- Let $\mathcal{C}_n^{\mathsf{cat}}$ be the cost of the call $\mathsf{cat}(s,t)$, for any t, where n is the size of s.
- Each rule yields an equation on their cost:

$$\mathcal{C}_0^{\mathsf{cat}} \overset{\alpha}{=} 1$$
$$\mathcal{C}_{n+1}^{\mathsf{cat}} \overset{\beta}{=} 1+\mathcal{C}_n^{\mathsf{cat}}.$$

- Together, they yield $\mathcal{C}_n^{\mathsf{cat}} = n+1$. Why?

The trick consists in making the difference of two successive terms in the series and then summing up these differences:

$$\mathcal{C}^{\mathsf{cat}}_{n+1} - \mathcal{C}^{\mathsf{cat}}_{n} = 1 \qquad \text{where } n \geqslant 0$$

$$\sum_{n=0}^{p} (\mathcal{C}^{\mathsf{cat}}_{n+1} - \mathcal{C}^{\mathsf{cat}}_{n}) = \sum_{n=0}^{p} 1 \qquad \text{where } p \geqslant 0$$

$$\mathcal{C}^{\mathsf{cat}}_{p+1} - \mathcal{C}^{\mathsf{cat}}_{0} = p+1$$

Replacing $p+1$ by n we finally get

$$\mathcal{C}^{\mathsf{cat}}_{n} - \mathcal{C}^{\mathsf{cat}}_{0} = n \qquad \text{where } n > 0$$

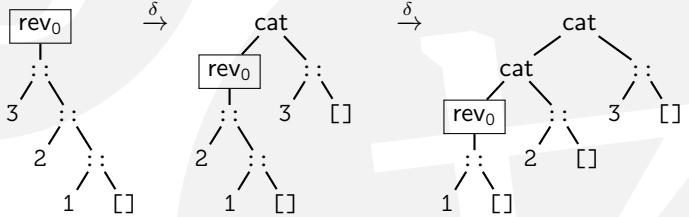
$$\mathcal{C}^{\mathsf{cat}}_{n} = n+1$$

Since that replacing n by 0 in the latter formula gives $\mathcal{C}^{\mathsf{cat}}_{0} = 0+1 = 1$, we can have only one formula for the cost: $\mathcal{C}^{\mathsf{cat}}_{n} = n+1$, where $n \geqslant 0$. □

- Let us consider the definition of a function rev_0 reversing a stack:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t \qquad \mathsf{rev}_0([]) \xrightarrow{\gamma} []$$
$$\mathsf{cat}(x::s,t) \xrightarrow{\beta} x::\mathsf{cat}(s,t) \qquad \mathsf{rev}_0(x::s) \xrightarrow{\delta} \mathsf{cat}(\mathsf{rev}_0(s),[x])$$

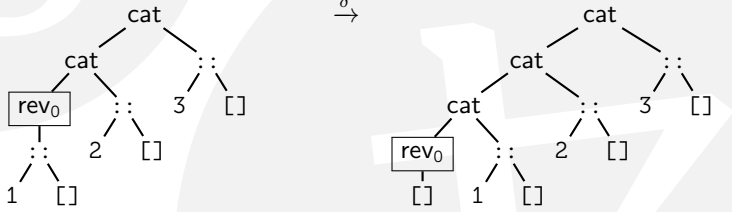
- The evaluation of $\mathsf{rev}_0([3;2;1])$ is shown with abstract syntax trees:



- Here are the definitions again:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t \qquad \mathsf{rev}_0([]) \xrightarrow{\gamma} []$$
$$\mathsf{cat}(x::s,t) \xrightarrow{\beta} x::\mathsf{cat}(s,t) \qquad \mathsf{rev}_0(x::s) \xrightarrow{\delta} \mathsf{cat}(\mathsf{rev}_0(s),[x])$$

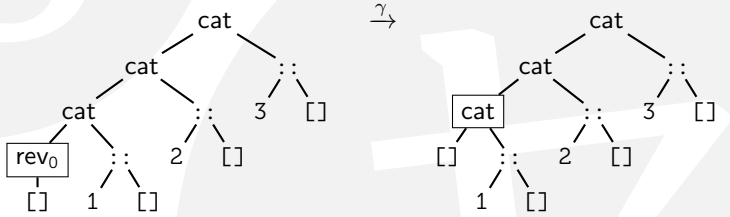
- Let us resume the evaluation of $\mathsf{rev}_0([3;2;1])$ from where we left:



- Here are the definitions again:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t \qquad \mathsf{rev}_0([]) \xrightarrow{\gamma} []$$
$$\mathsf{cat}(x::s,t) \xrightarrow[\beta]{} x::\mathsf{cat}(s,t) \qquad \mathsf{rev}_0(x::s) \xrightarrow[\delta]{} \mathsf{cat}(\mathsf{rev}_0(s),[x])$$

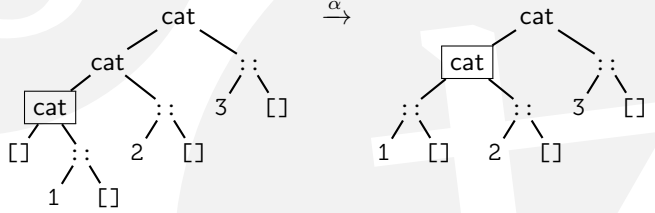
- Let us resume the evaluation of $\mathsf{rev}_0([3;2;1])$ from where we left:



- Here are the definitions again:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t \qquad \mathsf{rev}_0([]) \xrightarrow{\gamma} []$$
$$\mathsf{cat}(x::s,t) \xrightarrow{\beta} x::\mathsf{cat}(s,t) \qquad \mathsf{rev}_0(x::s) \xrightarrow{\delta} \mathsf{cat}(\mathsf{rev}_0(s),[x])$$

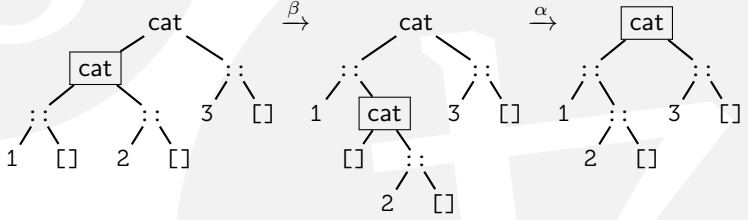
- Let us resume the evaluation of $\mathsf{rev}_0([3;2;1])$ from where we left:



- Here are the definitions again:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t \qquad \mathsf{rev}_0([]) \xrightarrow{\gamma} []$$
$$\mathsf{cat}(x::s,t) \xrightarrow{\beta} x::\mathsf{cat}(s,t) \qquad \mathsf{rev}_0(x::s) \xrightarrow{\delta} \mathsf{cat}(\mathsf{rev}_0(s),[x])$$

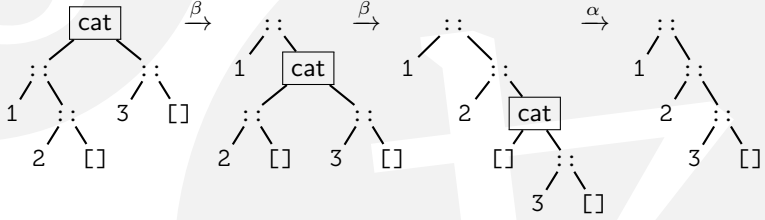
- Let us resume the evaluation of $\mathsf{rev}_0([3;2;1])$ from where we left:



- Here are the definitions again:

$$\mathsf{cat}([],t) \xrightarrow{\alpha} t \qquad \mathsf{rev}_0([]) \xrightarrow{\gamma} []$$
$$\mathsf{cat}(x::s,t) \xrightarrow{\beta} x::\mathsf{cat}(s,t) \qquad \mathsf{rev}_0(x::s) \xrightarrow{\delta} \mathsf{cat}(\mathsf{rev}_0(s),[x])$$

- Let us resume the evaluation of $\mathsf{rev}_0([3;2;1])$ from where we left:



- Here are the definitions again:

$$\begin{array}{cc} \mathsf{cat}([],t) \xrightarrow{\alpha} t & \mathsf{rev}_0([]) \xrightarrow{\gamma} [] \\ \mathsf{cat}(x::s,t) \xrightarrow[\beta]{} x::\mathsf{cat}(s,t) & \mathsf{rev}_0(x::s) \xrightarrow[\delta]{} \mathsf{cat}(\mathsf{rev}_0(s),[x]) \end{array}$$

- Let $\mathcal{C}_k^{\mathsf{rev}_0}$ be the cost of evaluating a call $\mathsf{rev}_0(l)$, where the list l has length k.
- Let $\mathcal{C}_k^{\mathsf{cat}}$ be the cost of evaluating a call $\mathsf{cat}(l)$, where the list l has length k.
- The definition of rev_0 directly leads to the recurrence equations

$$\mathcal{C}_0^{\mathsf{rev}_0} \overset{\gamma}{=} 1$$
$$\mathcal{C}_{k+1}^{\mathsf{rev}_0} \overset{\delta}{=} 1+\mathcal{C}_k^{\mathsf{rev}_0}+\mathcal{C}_k^{\mathsf{cat}} = \mathcal{C}_k^{\mathsf{rev}_0}+k+2$$

because the length of $\mathsf{rev}_0(s)$ is k if the length of s is k, and we already know $\mathcal{C}_k^{\mathsf{cat}} = k+1$.

- We have
$$\sum_{k=0}^{n-1}(\mathcal{C}_{k+1}^{\mathsf{rev}_0}-\mathcal{C}_k^{\mathsf{rev}_0}) = \mathcal{C}_n^{\mathsf{rev}_0}-\mathcal{C}_0^{\mathsf{rev}_0} = \sum_{k=0}^{n-1}(k+2) = 2n+\sum_{k=0}^{n-1}k.$$
- The remaining sum is a classic of algebra:
$$2\cdot\sum_{k=0}^{n-1}k = \sum_{k=0}^{n-1}k+\sum_{k=0}^{n-1}k = \sum_{k=0}^{n-1}k+\sum_{k=0}^{n-1}(n-k-1) = n(n-1).$$
- Consequently,
$$\sum_{k=0}^{n-1}k = \frac{1}{2}n(n-1).$$
- We can finally conclude
$$\mathcal{C}_n^{\mathsf{rev}_0} = \mathcal{C}_0^{\mathsf{rev}_0}+2n+\frac{1}{2}n(n-1) = \frac{1}{2}n^2+\frac{3}{2}n+1 \sim \frac{1}{2}n^2.$$

- Another way to reach the result is to induce an **evaluation trace**. A trace is a composition of rewrite rules, noted using the mathematical convention for multiplication.
- From the evaluation of $\mathsf{rev}_0([3;3;1])$, we infer the trace $\mathcal{T}_n^{\mathsf{rev}_0}$ of the evaluation of $\mathsf{rev}_0(s)$, where n is the length of s:

$$\mathcal{T}_n^{\mathsf{rev}_0} := \delta^n\gamma\alpha(\beta\alpha)...(\beta^{n-1}\alpha) = \delta^n\gamma\prod_{k=0}^{n-1}\beta^k\alpha.$$

- This reads: "Apply δ n times, then γ, then α etc."
- If we note $|\mathcal{T}_n^{\mathsf{rev}_0}|$ the length of $\mathcal{T}_n^{\mathsf{rev}_0}$, that is, the number of rule applications it contains, we expect to have the equations $|x| = 1$, for a rule x, and $|x \cdot y| = |x| + |y|$, for rules x and y.

- By definition of the cost:

$$\mathcal{C}_n^{\mathsf{rev}_0} := |\mathcal{T}_n^{\mathsf{rev}_0}| = \left|\delta^n\gamma\prod_{k=0}^{n-1}\beta^k\alpha\right| = |\delta^n\gamma| + \sum_{k=0}^{n-1}|\beta^k\alpha|$$

$$= (n+1) + \sum_{k=0}^{n-1}(k+1) = (n+1) + \sum_{k=1}^{n}k = \frac{1}{2}n^2 + \frac{3}{2}n + 1 \sim \frac{1}{2}n^2.$$

- Since we inferred our trace from an example, the cost function we derived may be wrong: we need to prove it by **induction**.
- Let $\mathsf{Quad}(n)$ be the property $\mathcal{C}_n^{\mathsf{rev}_0} = (n^2+3n+2)/2$. We already know that $\mathsf{Quad}(0)$ is true. Let us suppose $\mathsf{Quad}(n)$ is true for some value of n (induction hypothesis) and let us prove $\mathsf{Quad}(n+1)$.
- We know $\mathcal{C}_{n+1}^{\mathsf{rev}_0} = \mathcal{C}_n^{\mathsf{rev}_0} + n + 2$. The induction hypothesis implies
$$\mathcal{C}_{n+1}^{\mathsf{rev}_0} = (n^2+3n+2)/2 + n + 2 = ((n+1)^2 + 3(n+1) + 2)/2,$$
which is $\mathsf{Quad}(n+1)$. Therefore, the induction principle says that the cost we found is always correct. □

- Yet another way to determine the cost of rev_0 consists in first **guessing** that it is quadratic, that is, $\mathcal{C}_n^{\mathsf{rev}_0} = an^2+bn+c$, where a, b and c are unknowns.
- Since there are three coefficients, we only need three values of $\mathcal{C}_n^{\mathsf{rev}_0}$ to determine them, for instance $n=0,1,2$.
- Making some traces, we find
$$\mathcal{C}_0^{\mathsf{rev}_0}=1,\ \mathcal{C}_1^{\mathsf{rev}_0}=3 \text{ and } \mathcal{C}_2^{\mathsf{rev}_0}=6$$
so we solve the linear system
$$\mathcal{C}_0^{\mathsf{rev}_0}=c=1,\quad \mathcal{C}_1^{\mathsf{rev}_0}=a+b+c=3,\quad \mathcal{C}_2^{\mathsf{rev}_0}=a\cdot 2^2+b\cdot 2+c=6.$$
- Hence $a=1/2$, $b=3/2$ and $c=1$, that is, $\mathcal{C}_n^{\mathsf{rev}_0}=(n^2+3n+2)/2$.
- We may have been wrong with our guess of a quadratic cost: we need also here to prove it by **induction** exactly as before.

- We would be remiss not to consider a more efficient (that is, less costly) definition of a reversal function.
- Here it is:

$$\mathsf{rev}(s) \xrightarrow{\epsilon} \mathsf{rcat}(s,[])$$
$$\mathsf{rcat}([],t) \xrightarrow{\zeta} t$$
$$\mathsf{rcat}(x::s,t) \xrightarrow{\eta} \mathsf{rcat}(s,x::t)$$

(The name "rcat" stands for *reverse and catenate*.)

- The additional parameter introduced by "**rcat**" accumulates partial results, thus is called an **accumulator**:

$$\begin{aligned}\mathsf{rev}([3;2;1]) &\xrightarrow{\epsilon} \mathsf{rcat}([3;2;1],[])\\ &\xrightarrow{\eta} \mathsf{rcat}([2;1],[3])\\ &\xrightarrow{\eta} \mathsf{rcat}([1],[2;3])\\ &\xrightarrow{\eta} \mathsf{rcat}([],[1;2;3])\\ &\xrightarrow{\zeta} [1;2;3].\end{aligned}$$

- To draw the cost of **rev**, it is sufficient to notice that the first argument of **rcat** strictly decreases at each rewrite, so all evaluation traces have the shape $\epsilon\eta^n\zeta$, so $\mathcal{C}_n^{\mathsf{rev}} = n+2$.
- The cost is **linear**, so **rev** must always be used instead of rev_0.
- The reason for the inefficiency of rev_0 can be seen in the fact that rule δ produces a series of calls to "**cat**" following the pattern
$$\mathsf{rev}_0(s) \twoheadrightarrow \mathsf{cat}(\mathsf{cat}(\ldots\mathsf{cat}([],[x_n]),\ldots,[x_2]),[x_1]),$$
where $s = [x_1, x_2, \ldots, x_n]$.
- The cost of all these calls to "**cat**" is thus
$$1+2+\cdots+(n-1) = \tfrac{1}{2}n(n-1).$$
because the cost of $\mathsf{cat}(s,t)$ is $n+1$.
- The problem is not calling "**cat**", but the fact that the calls are embedded in the most unfavourable configuration. (Cats are not evil.)

Extremal costs

- When considering sorting programs based on comparisons, the cost varies depending on the algorithm and it also often depends on the original partial ordering of the keys.
- Therefore, size does not capture all aspects needed to assess efficiency.
- This quite naturally leads to consider bounds on the cost: for a given input size, we seek the configurations of the input that minimise and maximise the cost, respectively called the
  - **minimum cost** (cost for the **best case**) and
  - **maximum cost** (cost for the **worst case**).
- For example, some sorting algorithms have their worst case when the keys are already sorted, others when they are sorted in reverse order, etc.

Average cost

- Once we obtain bounds on a cost, the question about the **average** or **mean cost** (also called **expected cost**) arises as well.
- It is the arithmetic mean of the costs for all possible inputs of a given size.
- Some care is necessary, as there must be a finite number of such inputs.
- For instance, to assess the mean cost of sorting algorithms based on comparisons, it is usual to assume
  1. that the input is a series of n **distinct keys** and
  2. that the sum of the costs is taken over all their **permutations**, thus divided by $n!$, the number of permutations of size n.

Average cost

- The uniqueness constraint actually allows the analysis to equivalently, and more simply, consider the permutations of $(1,2,\ldots,n)$.
- Some sorting algorithms, like **merge sort** or **insertion sort**, have their average cost **asymptotically equivalent** to their maximum cost.
- In other words, for increasingly large numbers of keys, the ratio of the two costs become arbitrarily close to 1.
- Some others, like **Hoare's sort**, also known as **quicksort**, have the growth rate of their average cost being of a lower magnitude than the maximum cost, on an asymptotic scale.

Amortised cost

- Sometimes an update is costly because it is delayed by an imbalance in the data structure that calls for an immediate remediation, but this remediation itself may lead to a state such that subsequent operations are faster than if the costly update had not happen.
- Therefore, when considering a series of updates, it may be overly pessimistic to sum the maximum costs of all the operations considered in isolation.
- Instead, **amortised analysis** takes into account the interactions between updates, so a lower maximum bound on the cost is derived.
- Note that this kind of analysis is inherently different from the average case analysis, as its object is the composition of different functions instead of independent calls to the same function on different inputs. Amortised analysis is a worst case analysis of a sequence of updates.

Equivalence classes of terms w.r.t. evaluation

- We may remark that
$$\mathsf{cat}([1],[2;3;4]) \twoheadrightarrow [1;2;3;4] \twoheadleftarrow \mathsf{cat}([1;2],[3;4])$$
- It is enlightening to create **equivalence classes** of terms that are joinable, based on the **equivalence relation** $(\equiv)$ defined as: $a \equiv b$ if there exists a value v such that $a \twoheadrightarrow v$ and $b \twoheadrightarrow v$.
- For instance, above: $\mathsf{cat}([1],[2;3;4]) \equiv \mathsf{cat}([1;2],[3;4])$.
- The relation $(\equiv)$ is indeed an equivalence because it is
  - **reflexive**: $a \equiv a$;
  - **symmetric**: if $a \equiv b$, then $b \equiv a$;
  - **transitive**: if $a \equiv b$ and $b \equiv c$, then $a \equiv c$.
- If we want to prove equivalences with variables ranging over infinite sets, like
$$\forall s,t,u \in \mathcal{S}.\mathsf{cat}(s,\mathsf{cat}(t,u)) \equiv \mathsf{cat}(\mathsf{cat}(s,t),u)$$
we need some **induction principle**.

- We define a **well-founded order** on a set A as being a binary relation $(\succ)$ which does not have any **infinite descending chains**, to wit, there is no $a_0 \succ a_1 \succ \ldots$
- The **well-founded induction principle** then states that, for any predicate $\aleph$, $\forall a \in A.\aleph(a)$ is implied by $\forall a.(\forall b.a \succ b \Rightarrow \aleph(b)) \Rightarrow \aleph(a)$.
- Because there are no infinite descending chains, any subset $B \subseteq A$ contains minimal elements $M \subseteq B$, that is, there is no $b \in B$ such that $a \succ b$, if $a \in M$.
- In this case, proving by well-founded induction degenerates into proving $\aleph(a)$ for all $a \in M$.
- When $A = \mathbb{N}$, this principle is called **mathematical (complete) induction**.

- **Structural induction** is another particular case where $t \succ s$ holds if, and only if, s is a proper **subterm** of t, namely, the abstract syntax tree of s is included in the tree of t and $s \neq t$.
- Sometimes, a restricted form is enough. For instance, we can define $x :: s \succ s$, for any term x and any stack $s \in S$. (Both x and s are **immediate subterms** of $x :: s$.)
- There is no infinite descending chain since $[]$ is the unique minimal element of S: no s satisfies $[] \succ s$; so the basis is $t = []$ and $\forall t.(\forall s.t \succ s \Rightarrow \aleph(s)) \Rightarrow \aleph(t)$ degenerates into $\aleph([])$.
- In other words, when proving by induction on the length of a list, the **base case** is the empty list.

- Let us prove the **associativity** of the function "cat", which we express formally by
$$\mathsf{CatAssoc}(s,t,u) : \mathsf{cat}(s,\mathsf{cat}(t,u)) \equiv \mathsf{cat}(\mathsf{cat}(s,t),u)$$
where s, t and u are stacks.
- We apply the well-founded induction principle to the structure of s, so we must establish
  - the basis $\forall t,u \in S.\mathsf{CatAssoc}([],t,u)$ and
  - step $\forall s,t,u \in S.\mathsf{CatAssoc}(s,t,u) \Rightarrow \forall x \in T.\mathsf{CatAssoc}(x :: s,t,u)$.
- The base case is direct:
$$\mathsf{cat}([],\mathsf{cat}(t,u)) \xrightarrow{\alpha} \mathsf{cat}(t,u) \xleftarrow{\alpha} \mathsf{cat}(\mathsf{cat}([],t),u)$$
- Let us assume now $\mathsf{CatAssoc}(s,t,u)$, called the **induction hypothesis**.

Proving associativity

- Let us prove $\mathsf{CatAssoc}(x :: s, t, u)$, for any term x.
- The goal here is to use the rewrite system as an abstract machine to prove a property, with the timely help of the induction principle.
- Precisely, we want to rewrite each side of the equivalence we wish to prove until we either find the same term (equality), or we use the induction hypothesis (equivalence).
- Since we are going to work with equivalences, we should lift the call-by-value reduction strategy here, as it does not matter when arguments are evaluated, as long as their evaluation terminates.

Unification

- We may start with the left-hand side of $\mathsf{CatAssoc}(x::s,t,u)$, that is $\mathsf{cat}(x::s,\mathsf{cat}(t,u))$.
- We notice that this is **not** a value, as the call contains a reducible call, $\mathsf{cat}(t,u)$.
- When using functions to compute, we match a call containing only values against the patterns of the rules.
- Here, we would like to match a pattern against a pattern.
- This is more general than matching, because values are patterns, and is called **unification**.
- Given two patterns, we want to find whether we can find assignments to the variables of these so that, when they are substituted by their denotation, the two patterns become equal.

- Here, we would like to unify $\mathsf{cat}(x :: s, \mathsf{cat}(t,u))$ (called the **goal**) and $\mathsf{cat}(x :: s, t)$ (the pattern).
- The problem is that we have variables that are the same in the goal and the pattern (*x*, *s* and *t*).
- The case of $x :: s$ and $x :: s$ is harmless, but the unification of $\mathsf{cat}(t,u)$ and t is not.
- That is why it is best to rename the variables in the pattern so they are not found in the goal (this is the α-**renaming** of λ-calculus), and then try to unify them.
- To simplify things even more, we are going to rename *all* the variables in the definition of "cat".

- Let us try for example the following renaming

$$\mathsf{cat}([],b) \xrightarrow{\alpha} b$$
$$\mathsf{cat}(i :: a,b) \xrightarrow{\beta} i :: \mathsf{cat}(a,b)$$

- We want to unify the goal $\mathsf{cat}(x :: s,\mathsf{cat}(t,u))$ with the pattern $\mathsf{cat}(i :: a,b)$.
- The unification yields the equations $x = i$, $s = a$ and $\mathsf{cat}(t,u) = b$.
- Note that we do not obtain assignments, as with pattern matching, but **equations**. An assignment binds a term to a variable: it is an asymmetric relation, whereas an equation symmetrically relates two terms.
- Then we apply the set of equations from the unification, so the right-hand side of β, $i :: \mathsf{cat}(a,b)$, becomes $x :: \mathsf{cat}(s,\mathsf{cat}(t,u))$.

- We have now proved the first step below:

$$\begin{aligned}\mathsf{cat}(x::s,\mathsf{cat}(t,u)) &\xrightarrow{\beta} x::\mathsf{cat}(s,\mathsf{cat}(t,u))\\ &\equiv\ x::\mathsf{cat}(\mathsf{cat}(s,t),u) \qquad (\mathsf{CatAssoc}(s,t,u))\\ &\xleftarrow{\beta} \mathsf{cat}(x::\mathsf{cat}(s,t),u)\\ &\xleftarrow{\beta} \mathsf{cat}(\mathsf{cat}(x::s,t),u).\end{aligned}$$

- Note the use of the inductive hypothesis $\mathsf{CatAssoc}(s,t,u)$.
- This derivation above establishes that $\mathsf{CatAssoc}(x::s,t,u)$ holds.
- The induction principle entails that $\forall s,t,u \in S.\mathsf{CatAssoc}(s,t,u)$. □
- We have seen earlier that the cost of $\mathsf{cat}(s,t)$ is $|s|+1$, where $|s|$ is the length of s. So the cost of $\mathsf{cat}(s,\mathsf{cat}(t,u))$ is $|s|+|t|+2$, whereas the cost of $\mathsf{cat}(\mathsf{cat}(s,t),u)$ is $2\cdot|s|+|t|+2$.
- Thanks to our equivalence theorem, a compiler could replace the latter expression by the former to **optimise** the program.